CSE341 (UG2023) - Software Testing, Validation, and Verification project

Team Members:

1. Omar Ahmed Abou Raia 23P0100
2. Hossam Ossama Hussieny 23P0010
3. Mariam Ibrahim Assar 23P0123
4. Mariam Tarek Farouk 23P0214
5. Hassan sherif 23P0017
6. Habiba El-sabaa ali 23P0103

Table of Contents

1. Project Overview 3
2. System Scope and Assumptions 3
   2.1 System Scope 3
   2.2 Assumptions 3
3. Test Environment 3
4. Black-Box Testing 4
   4.1 Equivalence Partitioning 4
   4.3 Black-Box Test Cases 4
5 State-Based Testing 5
   5.1 State Transition Matrix 5
   5.2 Allowed and Illegal Actions 5
   5.3 State-Based Test Scenarios 5
6 UI Testing 6
   6.1 UI Functional Checklist 6
   6.2 UI Bug List (Example) 6
7 Integration Testing 6
   7.1 Integration Flow 6
   7.2 Integration Test Scenarios 6
   8.1 Control Flow Graphs (CFGs) 7
   8.2 Branch Coverage Statement 8
   8.3 White-Box Unit Tests 9
9. Test-Driven Development (TDD) 10
   9.1 Feature Description 10
   9.2 TDD Process Explanation 11
10. Requirements Traceability Matrix 11
11. Test Execution Summary 11
12. Risks and Limitations 11
13. Conclusion 12

1. Project Overview

This project focuses on designing and executing software tests for a simplified banking system. The system supports client account management, deposit and withdrawal operations, administrative actions, and state-based account transitions. The objective is to validate system correctness and reliability using black-box testing, white-box testing, state-based testing, UI testing, integration testing, and Test-Driven Development (TDD).

2. System Scope and Assumptions

2.1 System Scope

The system under test includes client account creation, deposit and withdrawal transactions, administrative state transitions, and GUI-based interaction. The system does not include external banking APIs, real database persistence, performance testing, or security testing.

2.2 Assumptions

- Account states are limited to Verified, Suspended, and Closed.
- Negative or zero transaction amounts are invalid.
- Withdrawals are not allowed for Suspended or Closed accounts.
- Deposits are not allowed for Closed accounts.
- GUI behavior reflects backend business rules.

3. Test Environment

Programming Language: Java

Testing Framework: JUnit 4

Platform: Desktop environment

GUI: Provided GUI mockups / HTML screens

Testing Type: Manual and automated testing

4. Black-Box Testing

4.1 Equivalence Partitioning

Operation	Valid Input	Invalid Input
deposit	amount > 0	amount ≤ 0
withdraw	amount ≤ balance	amount > balance
Account state	Verified	Suspended, Closed

4.2 Boundary Value Analysis

operation	Boundary Value	Expected Result
deposit	0	fail
deposit	1	success
withdraw	balance	success
withdraw	Balance+1	fail

4.3 Black-Box Test Cases

Test ID	input	state	Expected Output
1	deposit(-100)	Verified	false
2	deposit(200)	Verified	true
3	deposit(50)	Closed	false
4	withdraw(500)	Verified	false
5	withdraw(50)	Suspended	false
6	withdraw(50)	Verified	true

5 State-Based Testing

5.1 State Transition Matrix

Current state	Action state	Next state
Unverified	Verify	Verified
Verified	Violation	Suspended
Suspended	Appeal	Verified
Suspended	Admin Action	closed
closed	Any Action	closed

5.2 Allowed and Illegal Actions

State	Allowed actions	Illegal actions
Verified	Deposit, withdraw	-----
suspended	View only	Withdraw, transfer
closed	View only	Deposit, withdraw

5.3 State-Based Test Scenarios

Test id	state	action	Expected result
1	closed	deposit	fail
2	suspended	withdraw	fail
3	suspended	appeal	verified
4	verified	violation	suspended

6 UI Testing

6.1 UI Functional Checklist

UI element	condition	Expected behavior
Status label	Any state	Displays correct state
Deposit button	closed	disabled
Withdraw button	suspended	disabled
Input field	Negative value	Error message
Notification box	success	Confirmation shown

6.2 UI Bug List (Example)

Bug id	description
1	Withdraw button enabled in Suspended state
2	No error message for negative deposit
3	Status label not updated after appeal

7 Integration Testing

7.1 Integration Flow

When a user initiates a transaction through the GUI, the controller validates the input and invokes the appropriate account method. The account processes the request and returns the result, which is then reflected in the GUI through a success or error message.

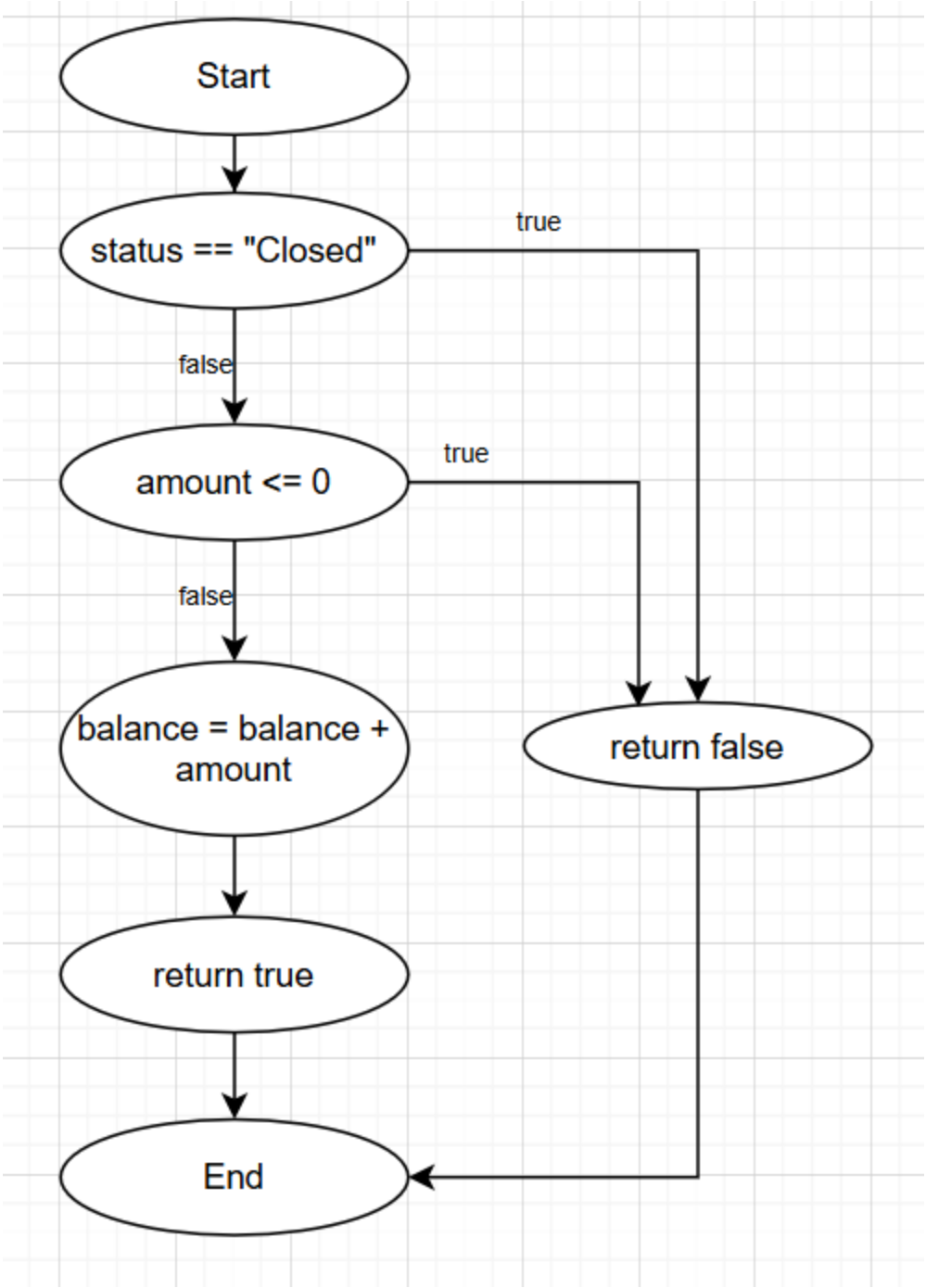
7.2 Integration Test Scenarios

Test Id	User Action	Expected Result
1	Valid deposit	Balance updated
2	Negative deposit	Error message
3	Withdraw in suspended	Action blocked
4	Overdraft withdrawal	Transaction rejected

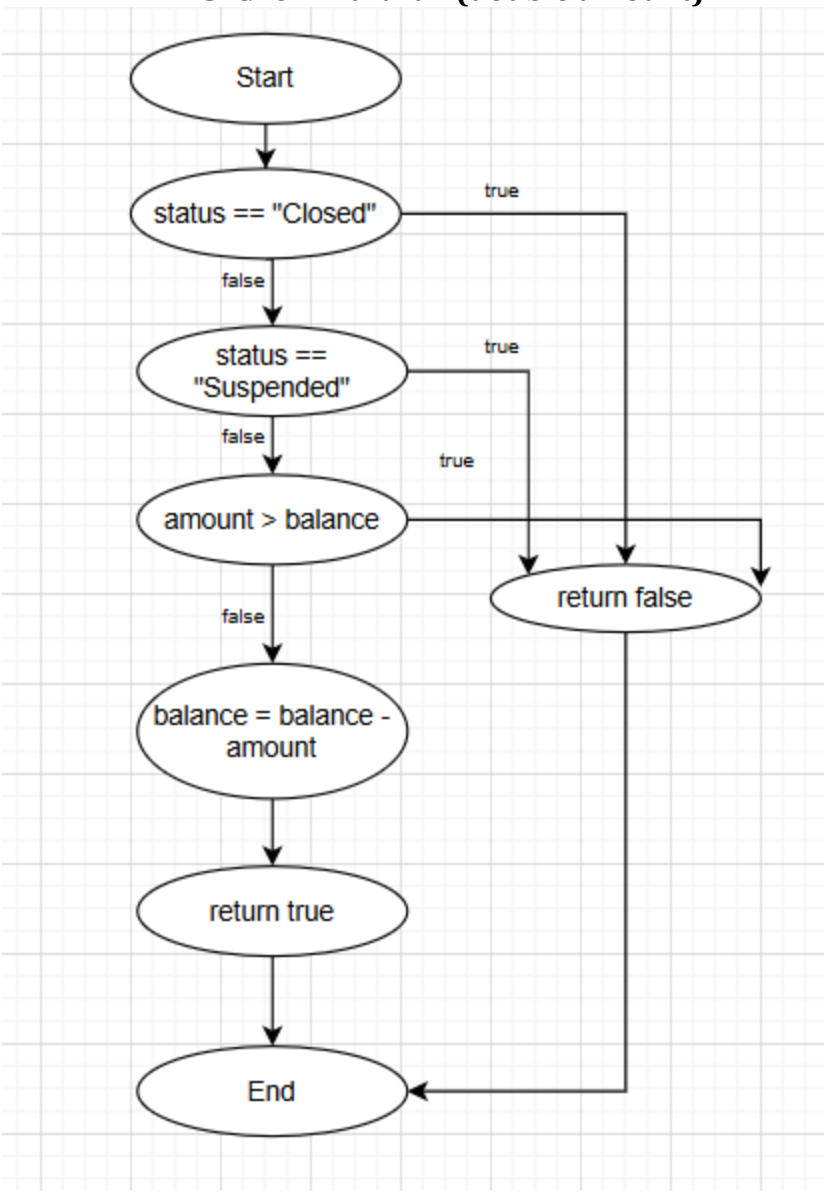
8. White-Box Testing

8.1 Control Flow Graphs (CFGs)

CFG for deposit(double amount)



CFG for withdraw(double amount)



8.2 Branch Coverage Statement

All decision paths represented in the Control Flow Graphs were exercised by unit tests, achieving 100% branch coverage for the deposit() and withdraw() methods.

8.3 White-Box Unit Tests

```java
import static org.junit.Assert.*;
import org.junit.Test;

public class AccountTest {

    // ===== White-Box Tests =====

    @Test
    public void testDepositClosed() {
        Account acc = new Account( initialBalance: 100,  initialStatus: "Closed");
        assertFalse(acc.deposit( amount: 50));
    }

    @Test
    public void testDepositZero() {
        Account acc = new Account( initialBalance: 100,  initialStatus: "Verified");
        assertFalse(acc.deposit( amount: 0));
    }

    @Test
    public void testDepositNegative() {
        Account acc = new Account( initialBalance: 100,  initialStatus: "Verified");
        assertFalse(acc.deposit( amount: -10));
    }

    @Test
    public void testDepositValid() {
        Account acc = new Account( initialBalance: 100,  initialStatus: "Verified");
        assertTrue(acc.deposit( amount: 50));
        assertEquals( expected: 150, acc.getBalance(),  delta: 0.01);
    }

    @Test
    public void testWithdrawSuspended() {
        Account acc = new Account( initialBalance: 100,  initialStatus: "Suspended");
        assertFalse(acc.withdraw( amount: 50));
    }
```

```java
    @Test
    public void testWithdrawClosed() {
        Account acc = new Account( initialBalance: 100,  initialStatus: "Closed");
        assertFalse(acc.withdraw( amount: 50));
    }

    @Test
    public void testWithdrawOverdraft() {
        Account acc = new Account( initialBalance: 100,  initialStatus: "Verified");
        assertFalse(acc.withdraw( amount: 200));
    }

    @Test
    public void testWithdrawValid() {
        Account acc = new Account( initialBalance: 200,  initialStatus: "Verified");
        assertTrue(acc.withdraw( amount: 100));
        assertEquals( expected: 100, acc.getBalance(),  delta: 0.01);
    }

    @Test
    public void testCreditScoreEligibility() {
        CreditScoreService service = new CreditScoreService();

        assertFalse(service.isEligible( creditScore: 400)); // should be false
        assertFalse(service.isEligible( creditScore: 599)); // below threshold

        assertTrue(service.isEligible( creditScore: 600));  // boundary
        assertTrue(service.isEligible( creditScore: 700));  // above threshold
    }
}
```

9. Test-Driven Development (TDD)

9.1 Feature Description

A credit score eligibility feature was added to determine whether a client qualifies for a loan based on a predefined credit score threshold.

9.2 TDD Process Explanation

Test-Driven Development was applied by first writing a failing unit test for the credit score eligibility feature. A minimal implementation was then added to satisfy the test, after which all tests passed successfully.

10. Requirements Traceability Matrix

Requirement	Testing Method
Valid deposit	Black-Box
Overdraft prevention	Black-Box
State restrictions	State-Based
UI restrictions	UI Testing
Internal logic	Whit-Box
New feature	TDD

11. Test Execution Summary

All designed test cases were executed successfully. Valid inputs produced expected outcomes, while invalid inputs and illegal state actions were correctly rejected. The final test execution completed with no failures.

```
PS C:\JUNIT_WORKSPACE> javac Account.java AccountTest.java CreditScoreService.java
PS C:\JUNIT_WORKSPACE> java org.junit.runner.JUnitCore AccountTest
JUnit version 4.13.2
.........
Time: 0.007

OK (9 tests)

PS C:\JUNIT_WORKSPACE> |
```

12. Risks and Limitations

- UI testing was partially manual.
- Integration testing was simulated.
- Performance and security testing were outside project scope.

13. Conclusion

This project successfully applied multiple testing techniques to validate the correctness and robustness of a banking system. The combined use of black-box, white-box, state-based, UI, integration testing, and TDD ensured comprehensive coverage and improved software quality.

Github Repo : https://github.com/HasanSherif77/Software_Testing